



ICS4U RST Step 2: Design Document



Click [HERE](#) for the Task Description

General Ideas Draft:

Overview - Describing Objects

General Structure Description:

The three main components of this program are the bank, player, and properties. The bank acts as the logic for the game, determining how the game plays out, and is also responsible for handling graphics. Methods such as start, rollDie, etc, enable it to respond to users' inputs and update the state of the game. In my code, such methods can generally be identified by the prefix handle, as these methods generally handle an event, say, a button is pushed. Along with this, the game handler also has methods, such as randomNumber generator, to support other methods to complete their roll, and various methods dedicated simply to rendering graphics, like showGameBoard, which renders the actual game. Datafields associated with the bank allow it to keep count of players, such as with the players array, store reference tiles on the board with the tiles array, designed to be an object array so that it can store objects of different classes (railroad, utilities and buildings), and finally the bank also has datafields for JavaFX components, like vbox which need to be dynamically updated. I also have my filereader and writer implemented in the Bank, as with this arrangement, I'm able to instantiate all the properties of the game of Monopoly by loading them from a text file.

Next, along with the Bank is the Player class. The Player class is the blueprint for all players of this monopoly game; it lays out what information is unique to each player. This includes their cash, position on the board, state of being in prison, state of being alive, and the associated methods that allow the Bank to see these states. This arrangement makes it easier to keep track of information associated with one individual, as it can be stored in one object and referenced as needed. The player class also stores a circle object, which can be referenced. It's used to represent the token that identifies the player's location on the board.

Finally, there are the properties. The property class acts as the blueprint/superclass for the unique purchasables in the game of the monopoly. All properties in Monopoly have a name, fee, state of being purchased, owner, inherent value for sale, and a monopoly level that impacts its rent. Hence, all these datafields are first laid out in the property class and extended to the more specific baseclasses. Additionally, the necessary setters and getters are added. One unique decision I made about the property class in this case was choosing to define it as abstract. This allows me to create abstract methods, like getFees, which must be uniquely implemented by each of the property's base classes. Railroads, utilities, and buildings all have unique fee structures. Hence, by making the property an abstract method, I can remind myself to implement methods, which are common to all of the three childclasses, but which are differently implemented for each.

In my program there are three base classes. The railroad, the utility, and the building. The railroad and utility are the most intuitive. Both the utility and the railroad have a fee structure



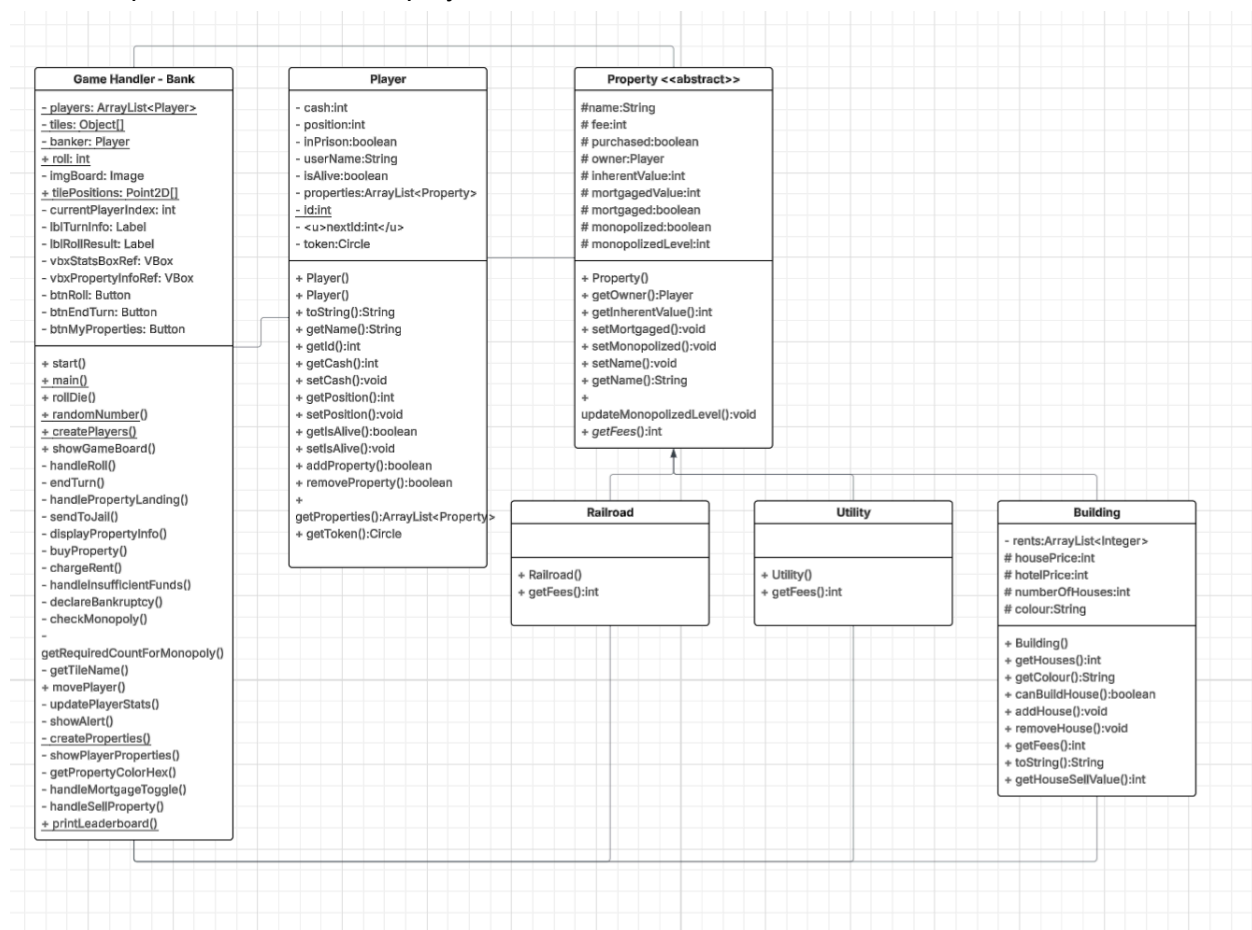
ICS4U RST Step 2: Design Document



linked to their monopoly level. In other words, the more that you own of these classes of properties, the more they charge for rent. For example, the rent for a railroad starts at 25, but doubles with each new railroad a player gains, doubling to 200. The utility is similar; it starts by charging four times the value of the die rolled, and increases to 10x when both utilities are owned. As such, these two properties don't require their own unique datafields; instead, they inherit them from the property superclass, and implement their own unique fee structure by defining the abstract method of `getFee` using inherited datafields like `monopolized level`. Hence, as can be demonstrated, this system of inheritance saves on redundancy.

Concerning the building, the building property must store a bit more, including its colour, so that monopolized levels can be gauged, and it must additionally store the cost of houses, as hotels cost the exact same, and an array to store the associated rents with upgraded properties.

Visual Representation of monopoly classes:





ICS4U RST Step 2: Design Document



In terms of algorithms, I've decided to use a linear search algorithm to search for the winning player during the game. This search algorithm will be called in between turns and will search for the top player. It will iterate through each player, counting all their assets together, and if their assets are the largest, they will be the leader by the end of the search algorithm, giving the current winner of the game of Monopoly.

Pseudocode:

FUNCTION findLeaderboard

 leader $\leftarrow$ null

 highestAssets $\leftarrow$ -1

 For each player p in players DO

 totalAssets $\leftarrow$ p.getCash()

 FOR each property prop in p.getProperties() DO

 totalAssets $\leftarrow$ totalAssets + prop.getInherentValue()

 IF prop is a Building THEN

 totalAssets $\leftarrow$ totalAssets + (prop.getHouses() * prop.housePrice)

 ENDIF

 ENDFOR

 IF totalAssets > highestAssets THEN

 highestAssets $\leftarrow$ totalAssets

 leader $\leftarrow$ p

 ENDIF

 ENDFOR

 RETURN leader, highestAssets

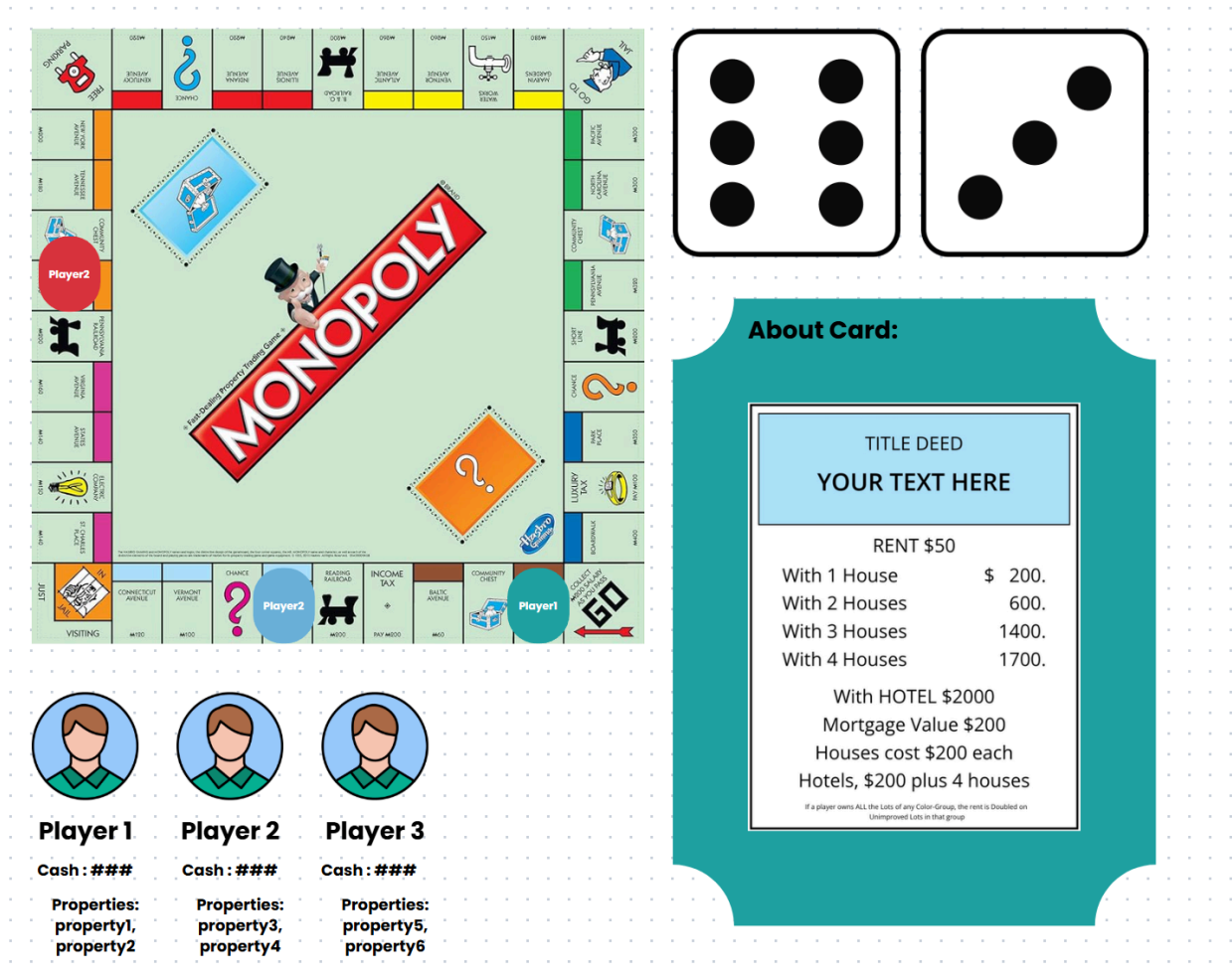
END FUNCTION



ICS4U RST Step 2: Design Document



GUI - MockUp



Concerns <i>Areas that need improvement</i>	Criteria	Strengths <i>Areas that exceed expectations</i>	Level
C.1 Demonstrate the ability to apply modular design concepts in computer programs			
	My design document uses UML properly to describe the objects my program will require. My design document describes at least one interesting algorithm my program will require. I have planned the user interface with mock ups. Weight: 15		



ICS4U RST

Step 2: Design Document

